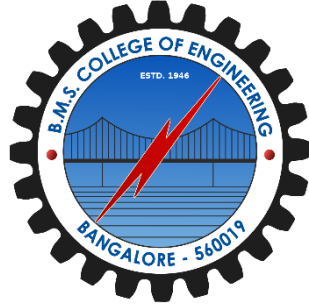




B.M.S. COLLEGE OF ENGINEERING

(Autonomous college under VTU)

Bull Temple Road, Basavanagudi, Bengaluru, Karnataka 560019

2024-2026

Department of Computer Applications

Report is submitted for fulfilment of Lab record in the subject

“Advance Java Programming”

(MMCE304)

Of

3rd Semester MCA

By

Madhu Shree G - 1BM24MC049

Under the Guidance of

Prof. R. V. Raghavendra Rao

Assistant Professor

Sno	Programs	Pgno
1	Create a Java Servlet that performs the following tasks: i) Provide an HTML form that accepts Username and Password as input fields. ii) Include two buttons on the form: Display – When clicked, the servlet should read the submitted username and password and display them on the browser. Clear – When clicked, the servlet should return the form with both input fields emptied. iii) Implement appropriate request handling in the servlet to distinguish between the two buttons and perform the corresponding action.	1-2
2	Develop a Java Servlet that enables users to download files stored on the server. The servlet should: i) Provide a list or option for users to select from multiple file types (such as .txt, .pdf, .jpg, .zip, etc.). ii) Detect the selected file type and set the appropriate MIME type dynamically. iii) Stream the requested file from the server to the client browser, prompting a file download dialog. iv) Ensure proper error handling for missing files or unsupported formats.	3-5
3	Design and develop a web application using JSP that allows a teacher to enter a student's performance scores and automatically compute the average and the corresponding grade. The application should: i) Provide a user-friendly webpage with form elements to accept multiple subject scores. ii) Validate the input values and ensure that only valid numeric scores are submitted. iii) Perform server-side processing to calculate the total, average, and assigned grade based on predefined criteria. iv) Display the calculated results clearly on the webpage.	6-7
4	Develop a web application for an HR portal to compute an employee's net salary based on the basic salary and allowance details. The application should perform the following tasks: i) Create a JSP page (salary.jsp) that accepts the following inputs from the user: • Employee Name • Basic Pay • Allowance Percentage ii) Implement a JavaBean class (SalaryBean.java) that includes methods to calculate: • DA (Dearness Allowance) • HRA (House Rent Allowance) • Net Salary iii) Display the calculated salary components and net salary on a separate results JSP page, using the JavaBean for data processing. iv) Ensure proper validation and clear presentation of input and output data.	8-11

5	Develop a web application for an insurance company to calculate the insurance premium based on the vehicle's model and age. The application must complete the following tasks: i) Create a JSP page (insurance.jsp) that accepts vehicle details from the user, including: • Car Make / Model • Manufacturing Year • Vehicle Price ii) Implement a JavaBean class (InsuranceBean.java) that applies appropriate business rules to compute the premium. The calculation should consider factors such as: • Older vehicles having higher premium rates • Price-based premium adjustments iii) Display the computed premium and a detailed cost breakdown on a separate JSP page (result.jsp). iv) Ensure clear presentation of input data and calculated results, along with proper validation of user inputs.	12-15
6	Create a JavaBean class named Product with the following attributes: • Product ID • Product Name • Product Price • Product Quantity The bean should include: 1) Appropriate getter and setter methods for all fields. 2) A method getTotalValue() that returns the total stock value calculated as: $\text{Total Value} = \text{Product Price} \times \text{Quantity Available}$ After creating the JavaBean: 3) Instantiate three Product objects, assign values to each object, and 4) Display all product details along with their total stock value.	16-19
7	Develop a menu-driven JDBC program to perform Create, Read, Update, Delete (CRUD) operations on a Customer table.	20-24
8	i) Use PreparedStatement to insert multiple rows into a table Books (bookId, title, author, price). Perform batch insertion. ii) Write a JDBC program to demonstrate transaction management for transferring money between two bank accounts. Use setAutoCommit(false), commit(), and rollback().	25-29
9	i) Create a Spring form for user registration and implement validation to ensure that the username field is not empty. ii) Demonstrate the use of @Valid and BindingResult in a Spring controller:	30-33
10	Create a Spring controller that displays "Welcome to Spring Boot" on accessing /welcome ii) Explain how form data is mapped to a Java bean using @ModelAttribute.	34-35
Part B		
1	Demonstration of Application: Employee management system	36-39

1. Create a Java Servlet that performs the following tasks:

i) Provide an HTML form that accepts Username and Password as input fields.

ii) Include two buttons on the form:

Display – When clicked, the servlet should read the submitted username and password and display them on the browser.

Clear – When clicked, the servlet should return the form with both input fields emptied.

iii) Implement appropriate request handling in the servlet to distinguish between the two buttons and perform the corresponding action.

LoginServlet.java

```
package com.lab;
```

```
import java.io.*;
```

```
import javax.servlet.*;
```

```
import javax.servlet.http.*;
```

```
import javax.servlet.annotation.WebServlet;
```

```
@WebServlet("/login")
```

```
public class LoginServlet extends HttpServlet {
```

```
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
```

```
        throws ServletException, IOException {
```

```
        String action = request.getParameter("action");
```

```
        response.setContentType("text/html");
```

```
        PrintWriter out = response.getWriter();
```

```
        if ("Display".equals(action)) {
```

```
            out.println("Username: " + request.getParameter("username") + "<br>");
```

```
        out.println("Password: " + request.getParameter("password"));
    } else {
        response.sendRedirect("index.html");
    }
}
}
```

index.html

```
<!DOCTYPE html>
<html>
<body>
<form action="login" method="post">
    Username: <input type="text" name="username"><br><br>
    Password: <input type="password" name="password"><br><br>
    <input type="submit" name="action" value="Display">
    <input type="submit" name="action" value="Clear">
</form>
</body>
</html>
```

Username:	Madhu Shree
Password:	
Display Clear	

Username: Madhu Shree
Password: Madhu@123

2. Develop a Java Servlet that enables users to download files stored on the server. The servlet should:

- i) Provide a list or option for users to select from multiple file types (such as .txt, .pdf, .jpg, .zip, etc.).**
- ii) Detect the selected file type and set the appropriate MIME type dynamically.**
- iii) Stream the requested file from the server to the client browser, prompting a file download dialog.**
- iv) Ensure proper error handling for missing files or unsupported formats.**

DownloadServlet.java

```
package com.lab;
```

```
import java.io.*;
```

```
import javax.servlet.*;
```

```
import javax.servlet.http.*;
```

```
import javax.servlet.annotation.WebServlet;
```

```
@WebServlet("/download")
```

```
public class DownloadServlet extends HttpServlet {
```

```
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
```

```
        throws ServletException, IOException {
```

```
        String fileName = request.getParameter("file");
```

```
        if (fileName == null) {
```

```
            response.getWriter().println("No file selected");
```

```
            return;
```

```
        }
```

```
// Access file from WEB-INF (secure location)

InputStream is = getServletContext()

    .getResourceAsStream("/WEB-INF/files/" + fileName);

if (is == null) {
    response.getWriter().println("File not found");
    return;
}

response.setContentType("application/octet-stream");
response.setHeader("Content-Disposition",
    "attachment; filename=\"" + fileName + "\"");

ServletOutputStream os = response.getOutputStream();

byte[] buffer = new byte[1024];
int bytesRead;
while ((bytesRead = is.read(buffer)) != -1) {
    os.write(buffer, 0, bytesRead);
}

is.close();
os.flush();
}
}
```

index.html

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2>Download File</h2>
```

```
<form action="download" method="get">
```

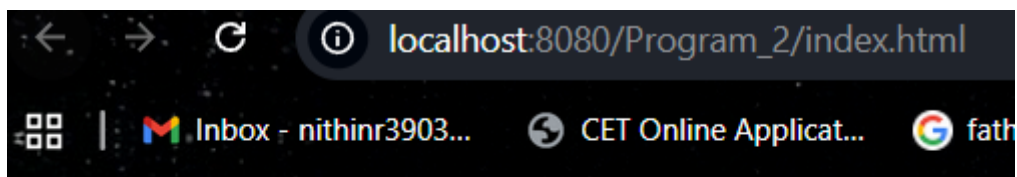
```
  <input type="hidden" name="file" value="sample.txt">
```

```
  <input type="submit" value="Download File">
```

```
</form>
```

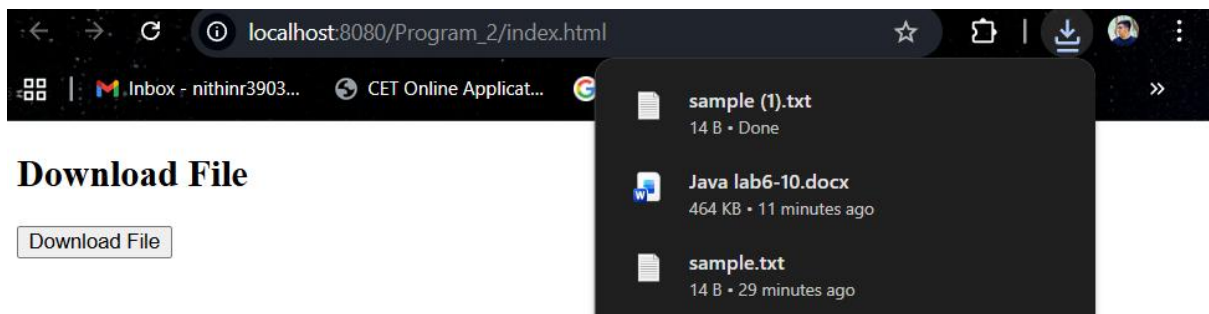
```
</body>
```

```
</html>
```



Download File

Download File



3. Design and develop a web application using JSP that allows a teacher to enter a student's performance scores and automatically compute the average and the corresponding grade. The application should:

- i) Provide a user-friendly webpage with form elements to accept multiple subject scores.**
- ii) Validate the input values and ensure that only valid numeric scores are submitted.**
- iii) Perform server-side processing to calculate the total, average, and assigned grade based on predefined criteria.**
- iv) Display the calculated results clearly on the webpage.**

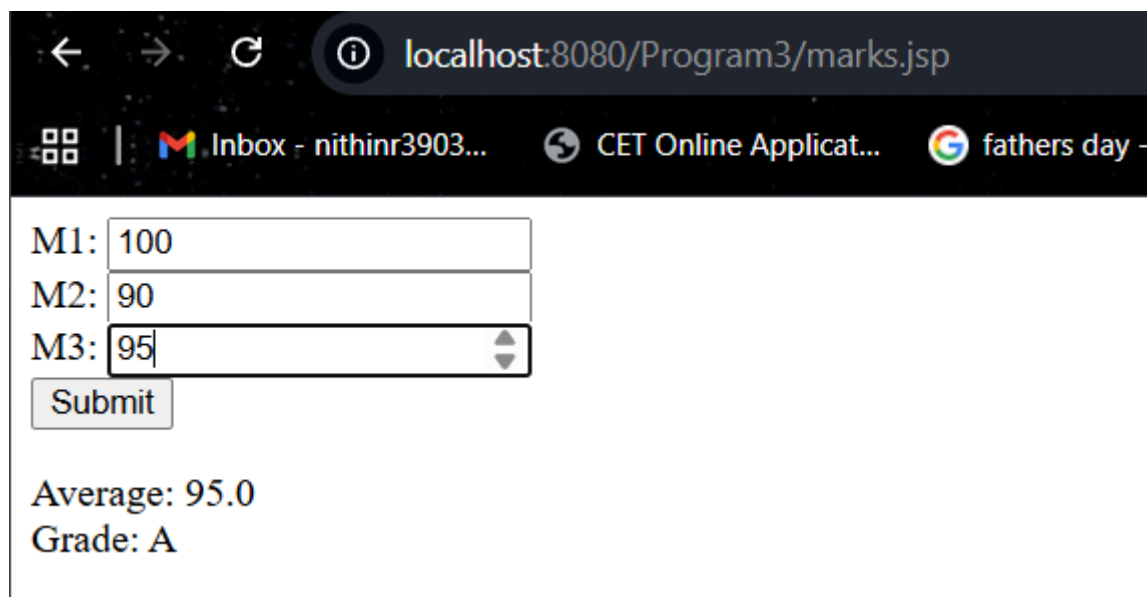
marks.jsp

```
<html>
<body>
<form method="post">
    M1: <input type="number" name="m1"><br>
    M2: <input type="number" name="m2"><br>
    M3: <input type="number" name="m3"><br>
    <input type="submit">
</form>

<%
if (request.getParameter("m1") != null) {
    int m1 = Integer.parseInt(request.getParameter("m1"));
    int m2 = Integer.parseInt(request.getParameter("m2"));
    int m3 = Integer.parseInt(request.getParameter("m3"));

    double avg = (m1 + m2 + m3) / 3.0;
    String grade = avg >= 75 ? "A" :
```

```
avg >= 60 ? "B" :  
avg >= 50 ? "C" : "Fail";  
%>  
Average: <%= avg %><br>  
Grade: <%= grade %>  
<% } %>  
</body>  
</html>
```



The screenshot shows a web browser window with the address bar displaying 'localhost:8080/Program3/marks.jsp'. The browser's tab bar shows three tabs: 'Inbox - nithinr3903...', 'CET Online Applicat...', and 'fathers day'. The main content area of the browser displays a web form with three input fields labeled 'M1:', 'M2:', and 'M3:'. The values entered are 100, 90, and 95 respectively. Below the input fields is a 'Submit' button. After clicking the button, the page displays the calculated 'Average: 95.0' and the resulting 'Grade: A'.

4. Develop a web application for an HR portal to compute an employee's net salary based

on the basic salary and allowance details. The application should perform the following tasks:

i) Create a JSP page (salary.jsp) that accepts the following inputs from the user:

- **Employee Name**
- **Basic Pay**
- **Allowance Percentage**

ii) Implement a JavaBean class (SalaryBean.java) that includes methods to calculate:

- **DA (Dearness Allowance)**
- **HRA (House Rent Allowance)**
- **Net Salary**

iii) Display the calculated salary components and net salary on a separate results JSP page, using the JavaBean for data processing.

iv) Ensure proper validation and clear presentation of input and output data.

SalaryBean.java

```
package com.lab;
```

```
public class SalaryBean {
```

```
    private String empName;
```

```
    private double basic;
```

```
    private double allowance;
```

```
    public void setEmpName(String empName) {
```

```
        this.empName = empName;
```

```
    }
```

```
public void setBasic(double basic) {  
    this.basic = basic;  
}  
  
public void setAllowance(double allowance) {  
    this.allowance = allowance;  
}  
  
public String getEmpName() {  
    return empName;  
}  
  
public double getDA() {  
    return basic * 0.10;  
}  
  
public double getHRA() {  
    return basic * 0.20;  
}  
  
public double getNetSalary() {  
    return basic + getDA() + getHRA() + allowance;  
}  
}
```

salary.jsp

```
<html>  
<body>
```

<h2>Employee Salary Details</h2>

<form action="result.jsp">

Employee Name:

<input type="text" name="empName">

Basic Pay:

<input type="number" name="basic">

Allowance:

<input type="number" name="allow">

<input type="submit" value="Calculate Salary">

</form>

</body>

</html>

result.jsp

<jsp:useBean id="s" class="com.lab.SalaryBean" />

<jsp:setProperty name="s" property="empName" param="empName" />

<jsp:setProperty name="s" property="basic" param="basic" />

<jsp:setProperty name="s" property="allowance" param="allow" />

<html>

<body>

<h2>Salary Result</h2>

Employee Name: <%= s.getEmpName() %>

DA: <%= s.getDA() %>

HRA: <%= s.getHRA() %>

Net Salary: <%= s.getNetSalary() %>

</body>

</html>

Employee Salary Details

Employee Name:

Basic Pay:

Allowance:

Salary Result

Employee Name: **Madhu Shree**

DA: 20000.0

HRA: 40000.0

Net Salary: **280000.0**

5. Develop a web application for an insurance company to calculate the insurance premium based on the vehicle's model and age. The application must complete the following tasks:

- i) Create a JSP page (insurance.jsp) that accepts vehicle details from the user, including:**
 - **Car Make / Model**
 - **Manufacturing Year**
 - **Vehicle Price**
- ii) Implement a JavaBean class (InsuranceBean.java) that applies appropriate business rules to compute the premium. The calculation should consider factors such as:**
 - **Older vehicles having higher premium rates**
 - **Price-based premium adjustments**
- iii) Display the computed premium and a detailed cost breakdown on a separate JSP page (result.jsp).**
- iv) Ensure clear presentation of input data and calculated results, along with proper validation of user inputs.**

InsuranceBean.java

```
package com.lab;
```

```
public class InsuranceBean {  
  
    private String model;  
    private int year;  
    private double price;  
  
    public void setModel(String model) {  
        this.model = model;  
    }  
}
```

```
}

public void setYear(int year) {
    this.year = year;
}

public void setPrice(double price) {
    this.price = price;
}

public String getModel() {
    return model;
}

public int getVehicleAge() {
    return 2025 - year;
}

public double getPremium() {
    int age = getVehicleAge();
    double premium = price * 0.05;

    if (age > 5) {
        premium += price * 0.02;
    }

    return premium;
}
}
```


insurance.jsp

```
<html>

<body>


<h2>Vehicle Insurance Details</h2>


<form action="result.jsp">

    Car Model:

    <input type="text" name="model"><br><br>


    Manufacturing Year:

    <input type="number" name="year"><br><br>


    Vehicle Price:

    <input type="number" name="price"><br><br>


    <input type="submit" value="Calculate Premium">

</form>


</body>

</html>
```

result.jsp

```
<jsp:useBean id="i" class="com.lab.InsuranceBean" />


<jsp:setProperty name="i" property="model" param="model" />
<jsp:setProperty name="i" property="year" param="year" />
<jsp:setProperty name="i" property="price" param="price" />
```

<html>

<body>

<h2>Insurance Premium Result</h2>

Vehicle Model: <%= i.getModel() %>

Vehicle Age: <%= i.getVehicleAge() %> years

Premium Amount: <%= i.getPremium() %>

</body>

</html>

Vehicle Insurance Details

Car Model:

Manufacturing Year:

Vehicle Price:

Insurance Premium Result

Vehicle Model: **Virtus GT**

Vehicle Age: 2 years

Premium Amount: **110000.0**

6.Create a JavaBean class named Product with the following attributes:

• Product ID • Product Name • Product Price • Product Quantity

The bean should include:

- 1) Appropriate getter and setter methods for all fields.**
- 2) A method getTotalValue() that returns the total stock value calculated as: Total Value = Product Price × Quantity Available After creating the JavaBean:**
- 3) Instantiate three Product objects, assign values to each object, and**
- 4) Display all product details along with their total stock value.**

Product.java

```
package com.example.product;

import java.io.Serializable;

public class Product implements Serializable {

    private int productId;

    private String productName;

    private double productPrice;

    private int productQuantity;

    public Product() {}

    public int getProductId() {

        return productId;

    }

    public void setProductId(int productId) {

        this.productId = productId;

    }

    public String getProductName() {

        return productName;

    }

    public void setProductName(String productName) {

        this.productName = productName;

    }

}
```

```
public double getProductPrice() {  
    return productPrice;  
}  
  
public void setProductPrice(double productPrice) {  
    this.productPrice = productPrice;  
}  
  
public int getProductQuantity() {  
    return productQuantity;  
}  
  
public void setProductQuantity(int productQuantity) {  
    this.productQuantity = productQuantity;  
}  
  
public double getTotalValue() {  
    return productPrice * productQuantity;  
}  
}
```

ProductTest.java

```
package com.example.product;  
  
public class ProductTest {  
    public static void main(String[] args) {  
        Product p1 = new Product();  
        p1.setProductId(101);  
        p1.setProductName("Laptop");  
        p1.setProductPrice(50000);  
        p1.setProductQuantity(2);  
        Product p2 = new Product();  
        p2.setProductId(102);  
        p2.setProductName("Mobile");  
        p2.setProductPrice(20000);  
    }  
}
```

```
p2.setProductQuantity(5);

Product p3 = new Product();

p3.setProductId(103);

p3.setProductName("Headphones");

p3.setProductPrice(3000);

p3.setProductQuantity(10);

display(p1);

display(p2);

display(p3);

}

static void display(Product p) {

    System.out.println("Product ID: " + p.getProductId());

    System.out.println("Product Name: " + p.getProductName());

    System.out.println("Price: " + p.getProductPrice());

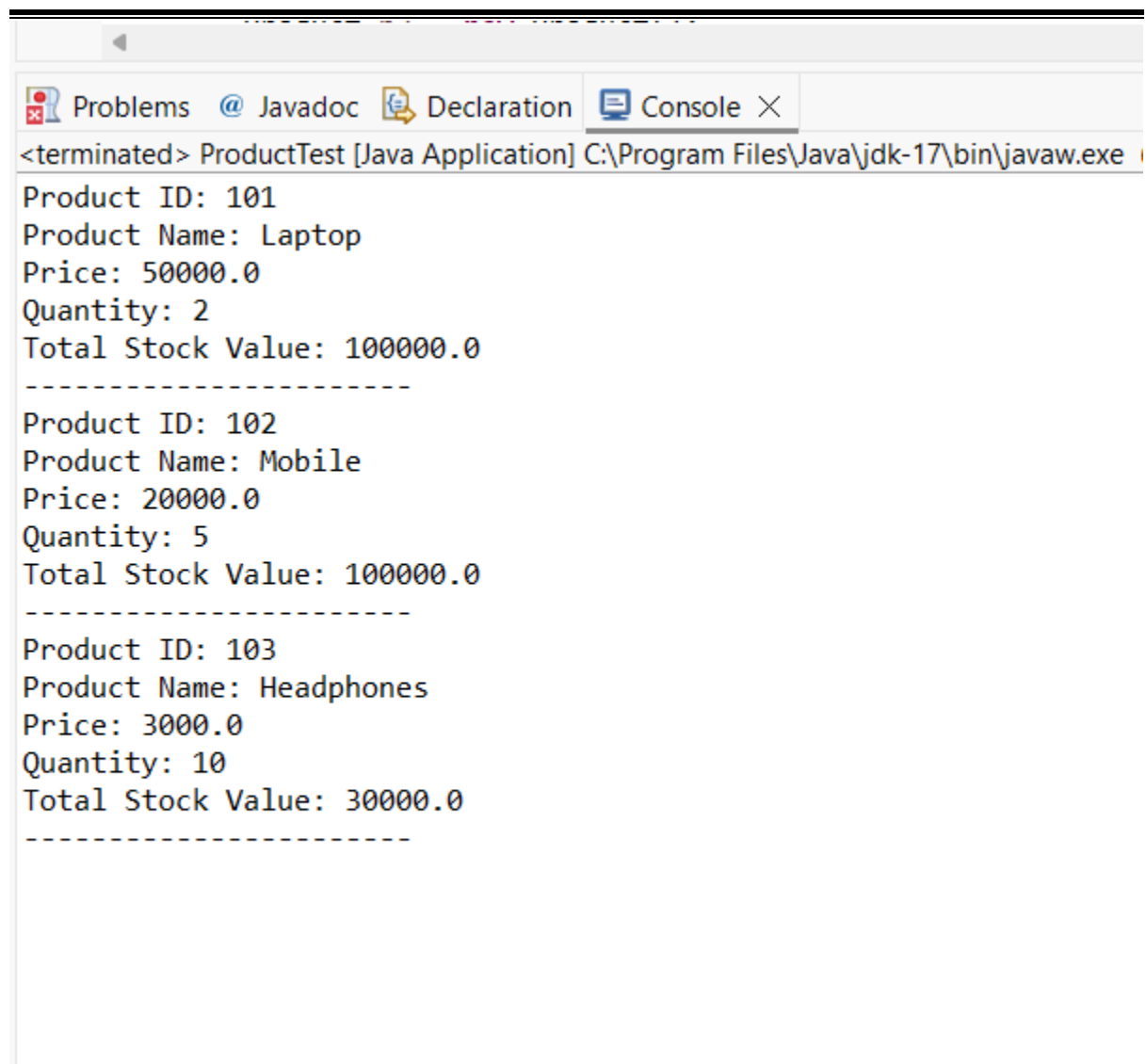
    System.out.println("Quantity: " + p.getProductQuantity());

    System.out.println("Total Stock Value: " + p.getTotalValue());

    System.out.println("-----");

}

}
```



```
<terminated> ProductTest [Java Application] C:\Program Files\Java\jdk-17\bin\javaw.exe
Product ID: 101
Product Name: Laptop
Price: 50000.0
Quantity: 2
Total Stock Value: 100000.0
-----
Product ID: 102
Product Name: Mobile
Price: 20000.0
Quantity: 5
Total Stock Value: 100000.0
-----
Product ID: 103
Product Name: Headphones
Price: 3000.0
Quantity: 10
Total Stock Value: 30000.0
-----
```

7.Develop a menu-driven JDBC program to perform Create, Read, Update, Delete (CRUD) operations on a Customer table.**customerCRUD.java**

```
package com.jdbc.customer;

import java.sql.*;
import java.util.Scanner;

public class CustomerCRUD {

    static Connection con;

    public static void main(String[] args) {

        try {

            Class.forName("com.mysql.cj.jdbc.Driver");

            con = DriverManager.getConnection(

                "jdbc:mysql://localhost:3306/jdbcdtb",

                "root",

                "Madhu@12");

            Scanner sc = new Scanner(System.in);

            int choice;

            do {

                System.out.println("\n--- CUSTOMER MENU ---");

                System.out.println("1. Insert Customer");

                System.out.println("2. View Customers");

                System.out.println("3. Update Customer");

                System.out.println("4. Delete Customer");

                System.out.println("5. Exit");

                System.out.print("Enter choice: ");

                choice = sc.nextInt();

                switch (choice) {

                    case 1:

                        insertCustomer(sc);
```

```
        break;

    case 2:
        viewCustomer();
        break;

    case 3:
        updateCustomer(sc);
        break;

    case 4:
        deleteCustomer(sc);
        break;

    case 5:
        System.out.println("Exiting...");
        break;

    default:
        System.out.println("Invalid choice");
    }
} while (choice != 5);
con.close();
} catch (Exception e) {
    e.printStackTrace();
}
}

static void insertCustomer(Scanner sc) throws Exception {
    PreparedStatement ps = con.prepareStatement(
        "INSERT INTO Customer VALUES (?, ?, ?)");

    System.out.print("Enter ID: ");
    ps.setInt(1, sc.nextInt());

    System.out.print("Enter Name: ");
    ps.setString(2, sc.next());
```



```
        System.out.print("Enter Email: ");

        ps.setString(3, sc.next());

        ps.executeUpdate();

        System.out.println("Customer Inserted Successfully");
    }

    static void viewCustomer() throws Exception {

        Statement st = con.createStatement();

        ResultSet rs = st.executeQuery("SELECT * FROM Customer");

        System.out.println("\nID  Name  Email");

        while (rs.next()) {

            System.out.println(

                rs.getInt(1) + " " +

                rs.getString(2) + " " +

                rs.getString(3)

            );

        }

    }

    static void updateCustomer(Scanner sc) throws Exception {

        PreparedStatement ps = con.prepareStatement(

            "UPDATE Customer SET email=? WHERE id=?");

        System.out.print("Enter New Email: ");

        ps.setString(1, sc.next());

        System.out.print("Enter Customer ID: ");

        ps.setInt(2, sc.nextInt());

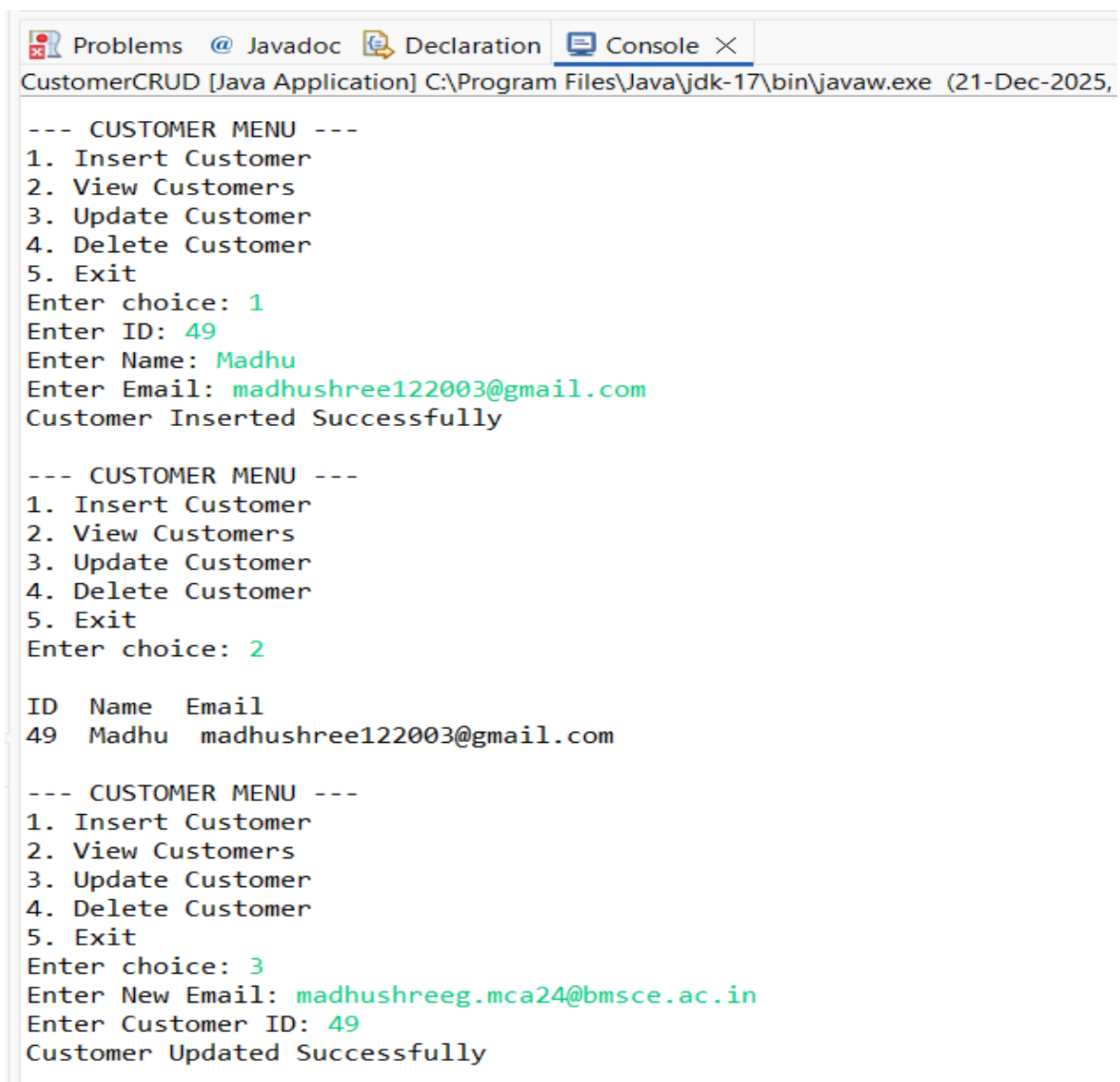
        ps.executeUpdate();

        System.out.println("Customer Updated Successfully");

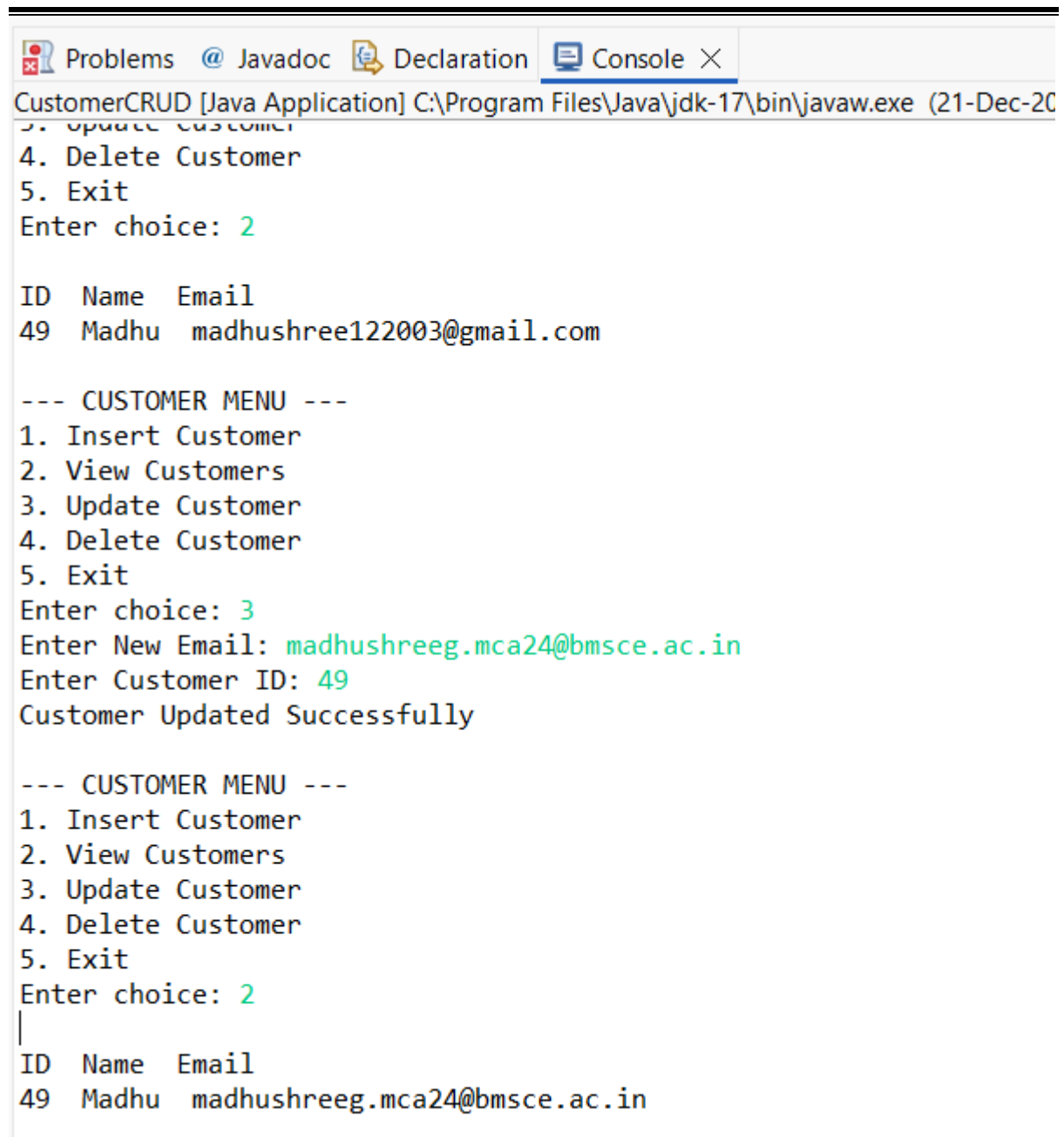
    }

    // DELETE
```

```
static void deleteCustomer(Scanner sc) throws Exception {  
    PreparedStatement ps = con.prepareStatement(  
        "DELETE FROM Customer WHERE id=?"  
    );  
    System.out.print("Enter Customer ID: ");  
    ps.setInt(1, sc.nextInt());  
    ps.executeUpdate();  
    System.out.println("Customer Deleted Successfully");  
}  
}
```



```
CustomerCRUD [Java Application] C:\Program Files\Java\jdk-17\bin\javaw.exe (21-Dec-2025,  
--- CUSTOMER MENU ---  
1. Insert Customer  
2. View Customers  
3. Update Customer  
4. Delete Customer  
5. Exit  
Enter choice: 1  
Enter ID: 49  
Enter Name: Madhu  
Enter Email: madhushree122003@gmail.com  
Customer Inserted Successfully  
  
--- CUSTOMER MENU ---  
1. Insert Customer  
2. View Customers  
3. Update Customer  
4. Delete Customer  
5. Exit  
Enter choice: 2  
  
ID   Name   Email  
49   Madhu   madhushree122003@gmail.com  
  
--- CUSTOMER MENU ---  
1. Insert Customer  
2. View Customers  
3. Update Customer  
4. Delete Customer  
5. Exit  
Enter choice: 3  
Enter New Email: madhushreeg.mca24@bmsce.ac.in  
Enter Customer ID: 49  
Customer Updated Successfully
```



```
CustomerCRUD [Java Application] C:\Program Files\Java\jdk-17\bin\javaw.exe (21-Dec-2020)
3. Update Customer
4. Delete Customer
5. Exit
Enter choice: 2

ID  Name  Email
49  Madhu  madhushree122003@gmail.com

--- CUSTOMER MENU ---
1. Insert Customer
2. View Customers
3. Update Customer
4. Delete Customer
5. Exit
Enter choice: 3
Enter New Email: madhushreeg.mca24@bmsce.ac.in
Enter Customer ID: 49
Customer Updated Successfully

--- CUSTOMER MENU ---
1. Insert Customer
2. View Customers
3. Update Customer
4. Delete Customer
5. Exit
Enter choice: 2

ID  Name  Email
49  Madhu  madhushreeg.mca24@bmsce.ac.in
```

8. i) Use PreparedStatement to insert multiple rows into a table Books (bookId, title, author, price). Perform batch insertion.

USE jdbcdb;

```
CREATE TABLE Books (  
    bookId INT PRIMARY KEY,  
    title VARCHAR(50),  
    author VARCHAR(50),  
    price DOUBLE  
);  
  
select *from Books;
```

BookBatchInsert.java

```
package com.jdbc.customer;  
  
import java.sql.Connection;  
import java.sql.DriverManager;  
import java.sql.PreparedStatement;  
  
public class BookBatchInsert {  
    public static void main(String[] args) {  
        try {  
            // Load driver  
            Class.forName("com.mysql.cj.jdbc.Driver");  
  
            // Connection  
            Connection con = DriverManager.getConnection(  
                "jdbc:mysql://localhost:3306/jdbcdb",  
                "root",  
                "Madhu@12");  
  
            // PreparedStatement  
            PreparedStatement ps = con.prepareStatement(  
                "INSERT INTO Books VALUES (?,?,?,?)");
```

```
// Book 1
ps.setInt(1, 1);
ps.setString(2, "Java Programming");
ps.setString(3, "James Gosling");
ps.setDouble(4, 550);
ps.addBatch();

// Book 2
ps.setInt(1, 2);
ps.setString(2, "Python");
ps.setString(3, "Guido");
ps.setDouble(4, 450);
ps.addBatch()

// Book 3
ps.setInt(1, 3);
ps.setString(2, "Database Systems");
ps.setString(3, "Elmasri");
ps.setDouble(4, 600);
ps.addBatch()

// Execute batch
ps.executeBatch();
System.out.println("Batch Insertion Completed");
con.close();
} catch (Exception e) {
    e.printStackTrace();
}
}
```

The screenshot shows two windows from an IDE. The top window, titled "Problems @ Javadoc Declaration Console X", displays the output of a Java application: "<terminated> BookBatchInsert [Java Application] C:\Program Files\Java\jdk-17 Batch Insertion Completed". The bottom window is a SQL editor with multiple tabs labeled "ebs*", "SQL File 5*", "SQL File 6*", "SQL File 7*", "SQL File 8*", and "SQL File 9*". The active tab shows SQL code:

```
2
3 CREATE TABLE Books (
4     bookId INT PRIMARY KEY,
5     title VARCHAR(50),
6     author VARCHAR(50),
7     price DOUBLE
8 );
9 select *from Books;
```

 Below the editor is a "Result Grid" showing the output of the SQL query. It has columns for bookId, title, author, and price. The data rows are: (1, Java Programming, James Gosling, 550), (2, Python, Guido, 450), (3, Database Systems, Elmasri, 600), and a row with NULL values. The grid also includes a "Filter Rows:" field and buttons for "Edit:", "Export/Import:", and "Limit to 1000 rows".

	bookId	title	author	price
▶	1	Java Programming	James Gosling	550
	2	Python	Guido	450
	3	Database Systems	Elmasri	600
*	NULL	NULL	NULL	NULL

8b)Write a JDBC program to demonstrate transaction management for transferring money between two bank accounts. Use setAutoCommit(false), commit(), and rollback().

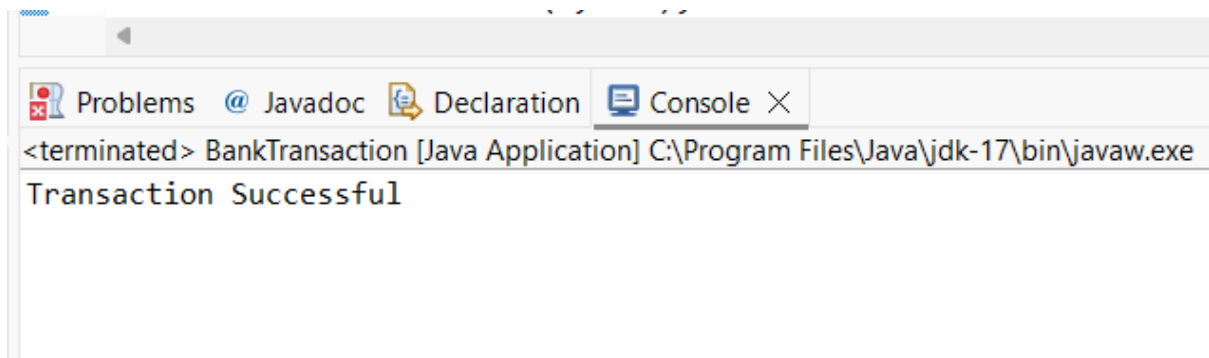
```
CREATE TABLE Bank (  
    acc_no INT PRIMARY KEY,  
    name VARCHAR(50),  
    balance DOUBLE
```

```
);  
  
INSERT INTO Bank VALUES (101, 'Madhu', 5000);  
  
INSERT INTO Bank VALUES (102, 'Manoj', 3000);  
  
select *from Bank;
```

BankTransaction.java

```
package com.jdbc.customer;  
  
import java.sql.Connection;  
  
import java.sql.DriverManager;  
  
import java.sql.PreparedStatement;  
  
public class BankTransaction {  
  
    public static void main(String[] args) {  
  
        try {  
  
            Class.forName("com.mysql.cj.jdbc.Driver");  
  
            Connection con = DriverManager.getConnection(  
  
                "jdbc:mysql://localhost:3306/jdbcdadb",  
  
                "root",  
  
                "Madhu@12");  
  
            // Disable auto-commit  
  
            con.setAutoCommit(false);  
  
            // Debit  
  
            PreparedStatement debit = con.prepareStatement(  
  
                "UPDATE Bank SET balance = balance - ? WHERE acc_no = ?");  
  
            debit.setDouble(1, 1000);  
  
            debit.setInt(2, 101);  
  
            debit.executeUpdate();  
  
            // Credit  
  
            PreparedStatement credit = con.prepareStatement(  
  
                "UPDATE Bank SET balance = balance + ? WHERE acc_no = ?");  
  
            credit.setDouble(1, 1000);
```

```
        credit.setInt(2, 102);  
        credit.executeUpdate();  
        // Commit transaction  
        con.commit();  
        System.out.println("Transaction Successful");  
        con.close();  
    } catch (Exception e) {  
        try {  
            System.out.println("Transaction Failed. Rolling Back");  
        } catch (Exception ex) {  
            ex.printStackTrace();  
        }  
    }  
}  
}
```



The screenshot shows the 'Result Grid' in a Java IDE. It displays a table with three columns: 'acc_no', 'name', and 'balance'. The table contains three rows: a header row, a row with '101', 'Madhu', and '4000', and a row with '102', 'Manoj', and '4000'. A third row with 'NULL', 'NULL', and 'NULL' is also visible.

	acc_no	name	balance
▶	101	Madhu	4000
	102	Manoj	4000
•	NULL	NULL	NULL

9. i) Create a Spring form for user registration and implement validation to ensure that the username field is not empty.

ii) Demonstrate the use of @Valid and BindingResult in a Spring controller:

User.java

```
package com.example.demo.model;

import jakarta.validation.constraints.NotEmpty;

public class User {

    @NotEmpty(message = "Username cannot be empty")
    private String username;

    public String getUsername() {
        return username;
    }

    public void setUsername(String username) {
        this.username = username;
    }
}
```

UserController.java

```
package com.example.demo.controller;

import com.example.demo.model.User;
import jakarta.validation.Valid;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.*;

@Controller
public class UserController {

    @GetMapping("/register")
    public String showForm(Model model) {
```

```
        model.addAttribute("user", new User());  
        return "register";  
    }  
    @PostMapping("/register")  
    public String submitForm(  
        @Valid @ModelAttribute("user") User user,  
        BindingResult result) {  
        if (result.hasErrors()) {  
            return "register";  
        }  
        return "success";  
    }  
}
```

register.html

```
<!DOCTYPE html>  
  
<html xmlns:th="http://www.thymeleaf.org">  
  
<head>  
    <title>User Registration</title>  
</head>  
  
<body>  
    <h2>User Registration</h2>  
    <form th:action="@{/register}" th:object="${user}" method="post">  
        Username:  
        <input type="text" th:field="*{username}" />  
        <span th:errors="*{username}" style="color:red"></span>  
        <br><br>  
        <input type="submit" value="Register"/>  
    </form>
```

</body>

</html>

success.html

<!DOCTYPE html>

<html>

<head>

<title>Success</title>

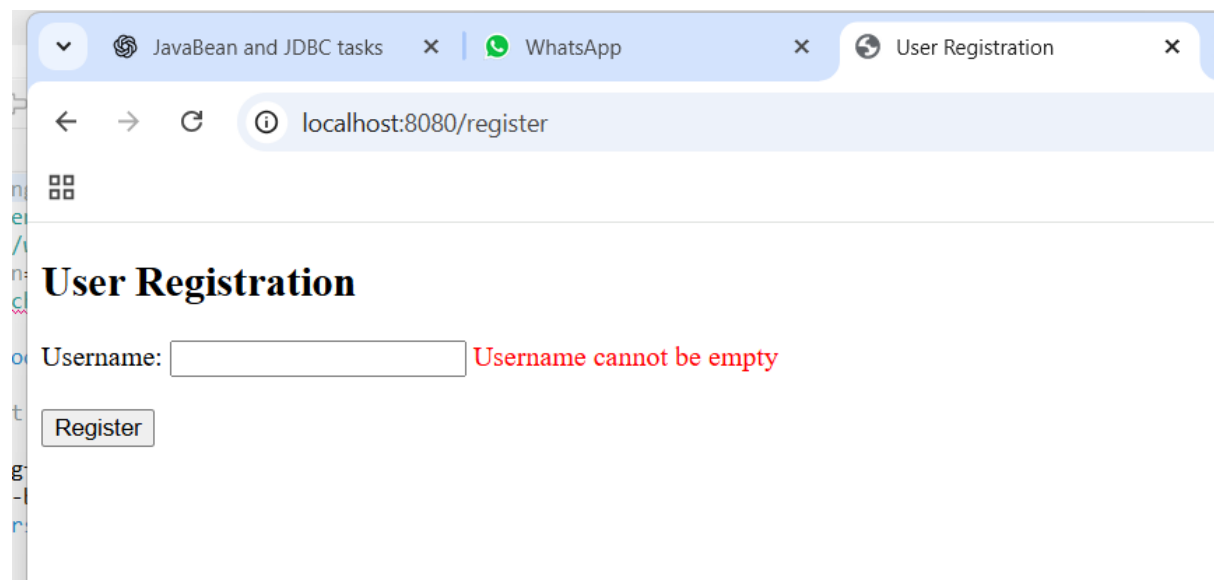
</head>

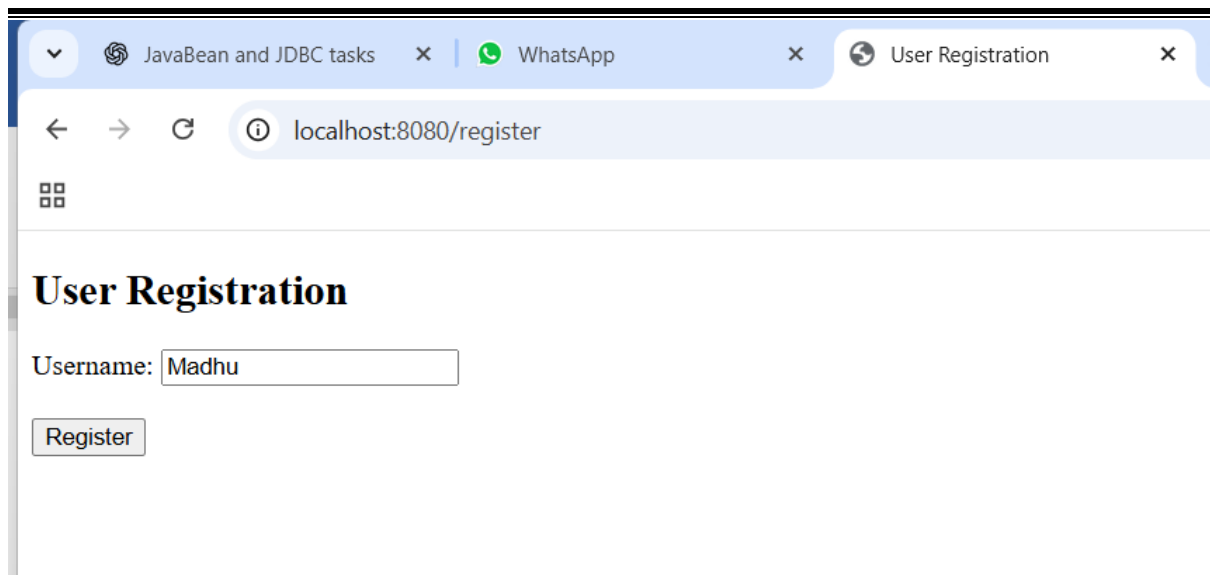
<body>

<h2>Registration Successful!</h2>

</body>

</html>



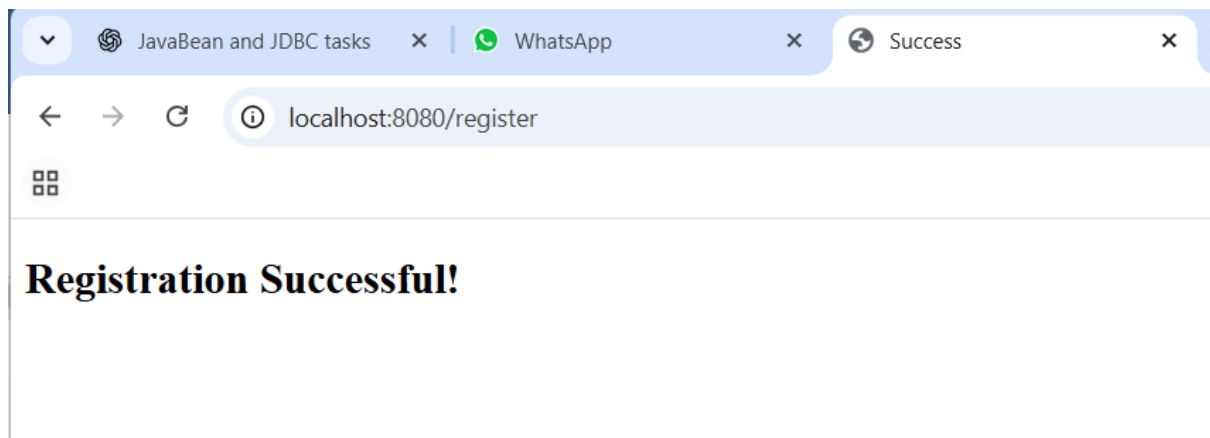


JavaBean and JDBC tasks x WhatsApp x User Registration x

localhost:8080/register

User Registration

Username:



JavaBean and JDBC tasks x WhatsApp x Success x

localhost:8080/register

Registration Successful!

10.Create a Spring controller that displays “Welcome to Spring Boot” on accessing /welcome ii) Explain how form data is mapped to a Java bean using @ModelAttribute.

```
package com.example.demo.controller;

import com.example.demo.model.User;
import jakarta.validation.Valid;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.ResponseBody;

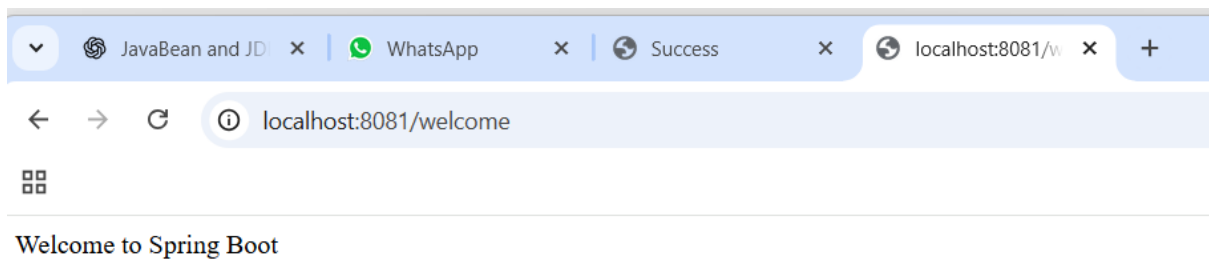
@Controller
public class UserController {

    @GetMapping("/welcome")
    @ResponseBody
    public String welcome() {
        return "Welcome to Spring Boot";
    }

    @GetMapping("/register")
    public String showForm(Model model) {
        model.addAttribute("user", new User());
        return "register";
    }

    @PostMapping("/register")
    public String submitForm(
        @Valid @ModelAttribute("user") User user,
        BindingResult result) {
```

```
        if (result.hasErrors()) {  
            return "register";  
        }  
        return "success";  
    }  
}
```



Part-B

1.To develop and demonstrate a working application based on the scenario they submitted earlier.

• Completion and demonstration of the application will be evaluated as part of the lab assessment.

Employee Management System

Problem Statement

In many organizations, employee information such as personal details, job roles, salaries, and contact information is often maintained using manual records or scattered digital files. This traditional approach leads to problems such as data redundancy, difficulty in updating records, lack of security, and inefficient retrieval of employee information.

Introduction

The **Employee Management System (EMS)** is a software application designed to manage and organize employee-related information in an efficient and systematic manner. It acts as a centralized platform where all employee data is stored in a database and can be accessed or modified through a user-friendly interface.

Technologies Used

Java (JSP + Servlets)

- To build dynamic web pages
- Handles request/response cycle
- Widely used in enterprise applications

MySQL Database

- Stores employee details
- Stores admin credentials
- Stores leave applications
- Lightweight & fast for CRUD operations

JDBC (Java Database Connectivity)

- Connects Java application with MySQL
- Allows executing SQL queries

Apache Tomcat Server

- Used to run JSP & Servlets
- Acts as the web server

HTML/CSS (Bootstrap / Custom CSS)

- To create responsive and professional UI
- Used gradients, animations, and glass-morphism for modern design

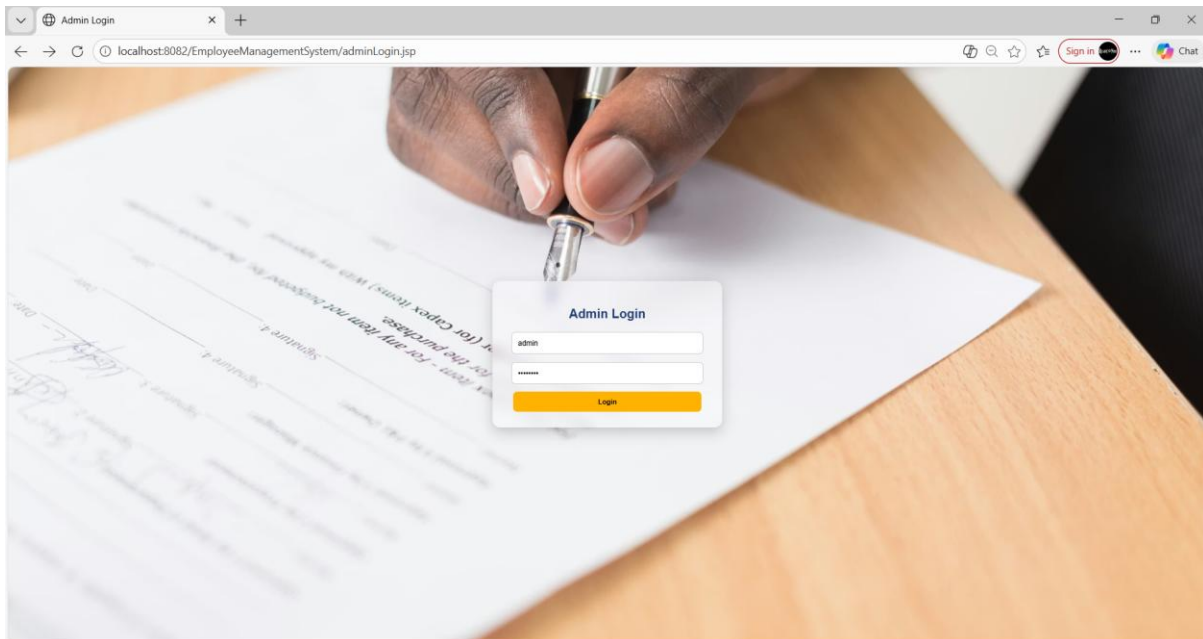


Fig1:Admin login page

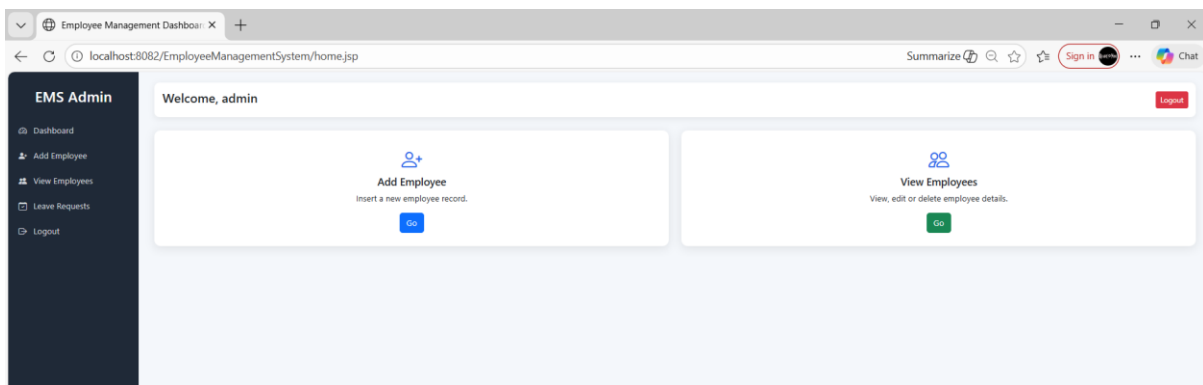


Fig2:Admin dashboard

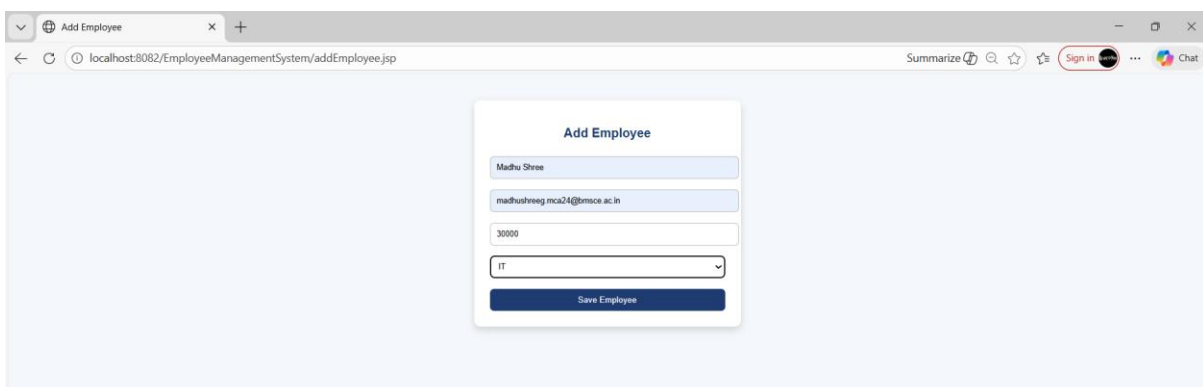
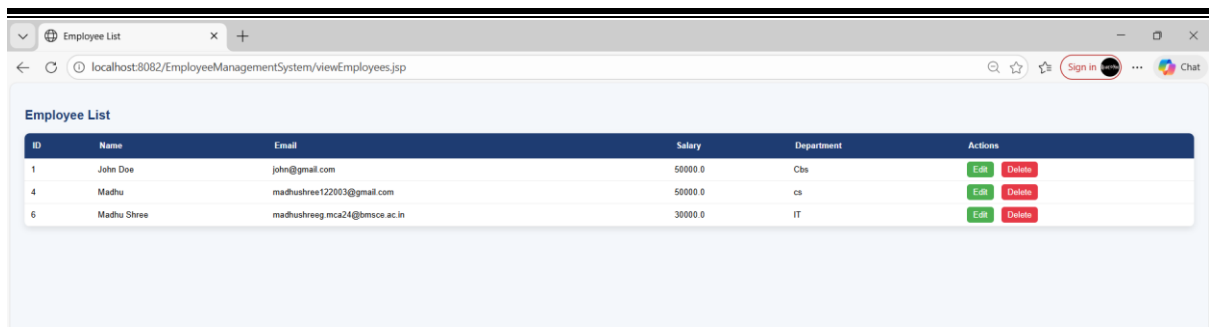
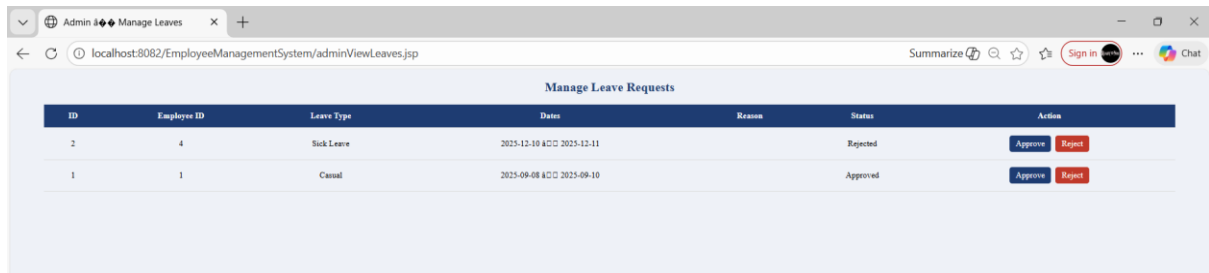


Fig3:admin add employee page



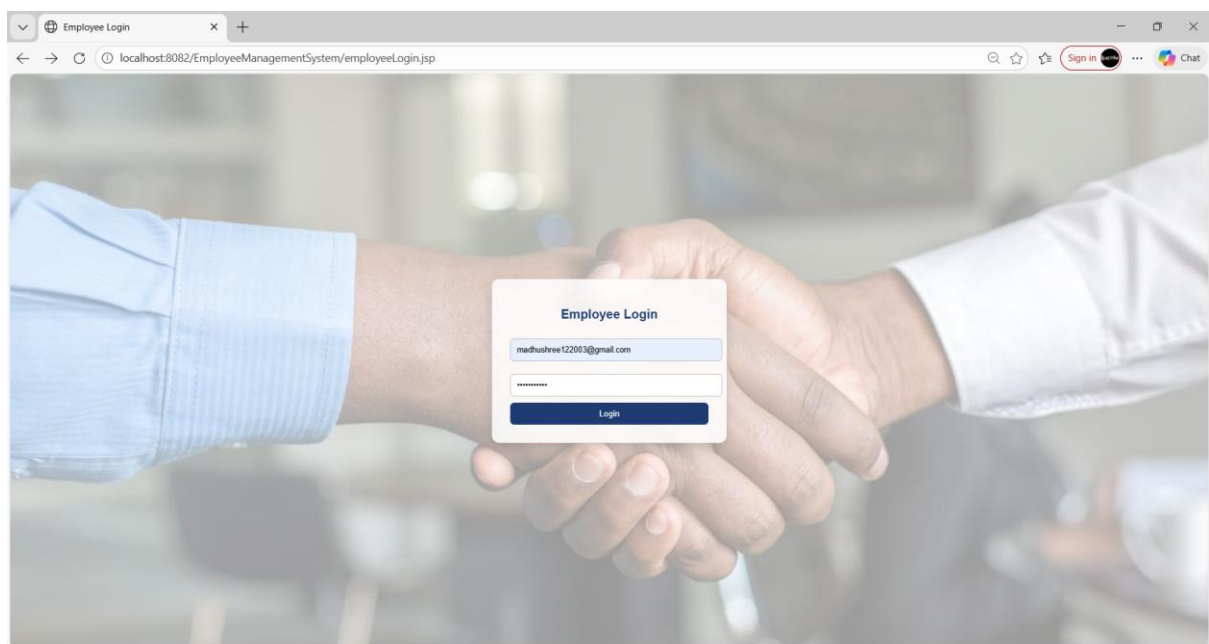
ID	Name	Email	Salary	Department	Actions
1	John Doe	john@gmail.com	50000.0	Cbs	Edit Delete
4	Madhu	madhushree122003@gmail.com	50000.0	cs	Edit Delete
6	Madhu Shree	madhushraeg_mca24@bmsce.ac.in	30000.0	IT	Edit Delete

Fig4:Admin view employee page



ID	Employee ID	Leave Type	Dates	Reason	Status	Action
2	4	Sick Leave	2025-12-10 to 2025-12-11		Rejected	Approve Reject
1	1	Casual	2025-09-08 to 2025-09-10		Approved	Approve Reject

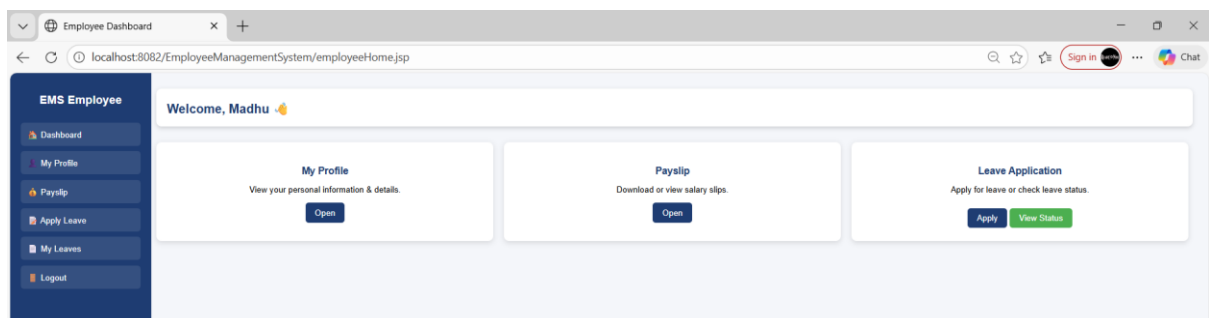
Fig5:Admin manage leave page



Employee Login

[Login](#)

Fig6:Employee login page



EMS Employee

Welcome, Madhu 🌟

My Profile

View your personal information & details.

[Open](#)

Payslip

Download or view salary slips.

[Open](#)

Leave Application

Apply for leave or check leave status.

[Apply](#) [View Status](#)

Fig7:Employee dashboard

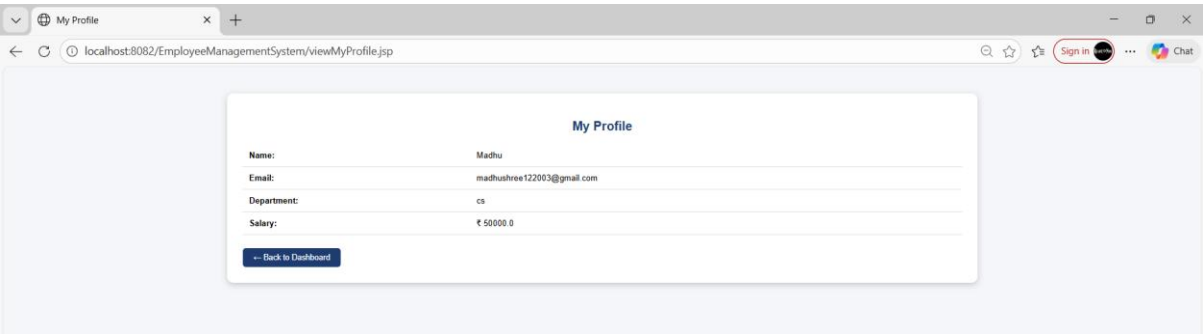


Fig8:Employee profile page

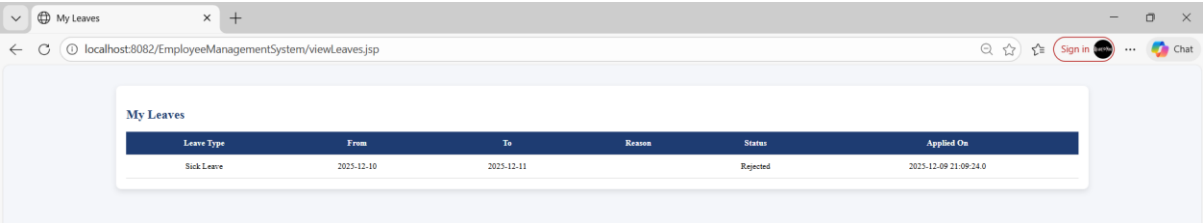


Fig9:Employee leave application page

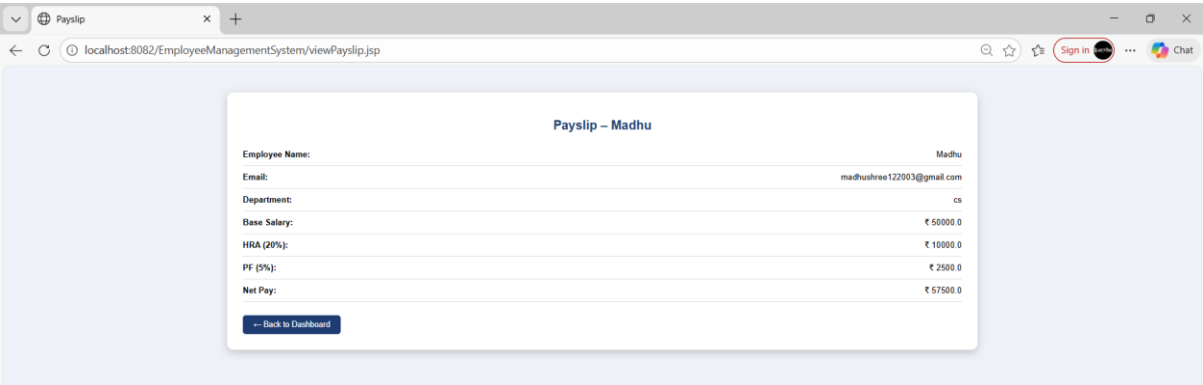


Fig10:Employee payslip page